MODULE 03 | TEXT PROCESSING

Text Processing

Slice, search, transform and summarise text with small tools chained together using pipes.

Mission	Topic	Commands / Concepts
1	Count and Pipe: wc, tee and the Pipeline	wc, |, tee, tr
2	Search: grep	grep, -i -v -n -c -r -E -w -A -B
3	Slice and Sort: cut, sort, uniq	cut, sort, uniq, uniq -c
4	Edit Streams: sed	sed, s///, -n, -i, d, p
5	Programmable Text: awk	awk, $1, NR, NF, -F, BEGIN, END
Boss	Log Detective	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

The Unix philosophy

Each tool does **one** job well. You combine tools with the pipe `|`, which sends the output of one command into the input of the next.

```
$ cat access.log | grep 404 | cut -d' ' -f1 | sort | uniq -c | sort -rn | head
   ^ read     ^ keep 404 lines  ^ take IP   ^ group  ^ count  ^ rank   ^ top 10
```

Streams

Stream	Number	Default	Redirect with
stdin	0	keyboard	<
stdout	1	screen	> or >>
stderr	2	screen	2>

```
$ command > out.txt          # stdout to file
$ command 2> errors.txt       # stderr to file
$ command > all.txt 2>&1      # both into one file
$ command | tee out.txt       # show on screen AND save
```

Practice data

Create this sample file. Examples in the guide use it.

```
$ cat > people.csv << 'EOF'
name,dept,salary,city
Asha,IT,52000,Chennai
Ravi,HR,41000,Coimbatore
Meena,IT,61000,Chennai
Karthik,Sales,45000,Madurai
Divya,IT,58000,Coimbatore
Ravi,HR,41000,Coimbatore
EOF
```

MISSION 1 | +50 XP

Count and Pipe: wc, tee and the Pipeline

Goal: Measure text and connect commands with pipes.

wc - word count

```
$ wc people.csv
  7  7 138 people.csv        # lines, words, bytes
$ wc -l people.csv           # lines only
$ wc -w people.csv           # words
$ wc -c people.csv           # bytes
$ wc -m people.csv           # characters
$ ls | wc -l                 # how many items in this folder?
```

Pipes in action

```
$ ls -l | head -5
$ history | tail -3
$ ps aux | wc -l            # how many processes?
$ cat people.csv | wc -l
```

tr - translate characters

```
$ echo "hello" | tr 'a-z' 'A-Z'      # HELLO
$ echo "a,b,c" | tr ',' '\n'         # one per line
$ echo "aabbcc" | tr -s 'a-c'         # squeeze repeats: abc
$ echo "h3ll0" | tr -d '0-9'         # delete digits: hll
```

tee - split the stream

```
$ ls -l | tee listing.txt | wc -l    # save AND count
$ echo "line" | tee -a log.txt        # -a appends
```

> **Tip**
> `cat file | cmd` works, but many commands take a filename directly (`wc -l file`). The shorter form is usually better.

Practice

- Count how many files are in `/etc` using `ls` and `wc`.
- Convert `people.csv` to upper case and save to `upper.csv`.
- Save the output of `ls -l /etc` to `etc.txt` and also print how many lines it has.

MISSION 2 | +50 XP

Search: grep

Goal: Find lines that match a word or pattern.

grep basics

```
$ grep IT people.csv                # lines containing IT
$ grep -i chennai people.csv         # ignore case
$ grep -v IT people.csv              # lines NOT containing IT
$ grep -n Ravi people.csv            # show line numbers
$ grep -c IT people.csv              # count matching lines
$ grep -w IT people.csv              # whole word only
$ grep -r "TODO" ~/lab              # search inside all files recursively
$ grep -l "error" *.log              # only list names of matching files
```

Option	Meaning
`-i`	Ignore case
`-v`	Invert: show non-matching lines
`-n`	Show line numbers
`-c`	Count matches
`-r`	Recursive through folders
`-w`	Match whole words
`-A n` / `-B n` / `-C n`	Show n lines After / Before / around each match
`-E`	Extended regex (same as `egrep`)
`-o`	Print only the matched part

Patterns (regular expressions)

Pattern	Matches
`^abc`	Line starting with abc
`abc$`	Line ending with abc
`a.c`	a, any one character, c
`a*`	Zero or more a
`[0-9]`	Any digit
`[A-Z][a-z]+`	Capitalised word (with `-E`)
`cat|dog`	Either word (with `-E`)

```
$ grep -E "^(Asha|Ravi)" people.csv     # lines starting with Asha or Ravi
$ grep -E "[0-9]{5}" people.csv          # 5-digit numbers
$ ps aux | grep ssh                      # search command output
$ grep -C 2 "error" app.log              # match plus 2 lines of context
```

Practice

- List employees from Chennai using grep.
- Count lines that do not contain HR.
- Find every line in `/etc/passwd` that starts with `root`.

MISSION 3 | +50 XP

Slice and Sort: cut, sort, uniq

Goal: Extract columns, order lines and find duplicates or frequencies.

cut - pick columns

```
$ cut -d',' -f1 people.csv          # first column (delimiter = comma)
$ cut -d',' -f1,3 people.csv        # columns 1 and 3
$ cut -d',' -f2-4 people.csv        # range
$ cut -c1-5 people.csv              # characters 1 to 5
$ cut -d: -f1 /etc/passwd           # usernames
```

sort - order lines

```
$ sort names.txt                    # alphabetical
$ sort -r names.txt                 # reverse
$ sort -n nums.txt                  # numeric (10 after 9, not after 1)
$ sort -h sizes.txt                 # human sizes (2K, 1M)
$ sort -u names.txt                 # unique
$ sort -t',' -k3 -n people.csv      # by 3rd column, numeric
$ sort -t',' -k3 -nr people.csv     # highest salary first
```

uniq - collapse duplicates

`uniq` only removes **adjacent** duplicates, so always `sort` first.

```
$ sort names.txt | uniq             # distinct values
$ sort names.txt | uniq -c          # count each
$ sort names.txt | uniq -d          # only duplicated lines
$ sort names.txt | uniq -u          # only lines that appear once
```

Classic frequency pattern

```
$ cut -d',' -f2 people.csv | tail -n +2 | sort | uniq -c | sort -rn
      3 IT
      2 HR
      1 Sales
```

Tip
`sort | uniq -c | sort -rn` is the most reusable one-liner in text processing. It answers 'what appears most often?'.

Practice

- Print only the `city` column of `people.csv` without the header.
- List each distinct city with its number of employees.
- Show the three highest salaries.

MISSION 4 | +50 XP

Edit Streams: sed

Goal: Find and replace or delete lines without opening an editor.

Substitute

The pattern is `s/old/new/flags`. Without `g`, only the first match per line is replaced.

```
$ echo "cat cat" | sed 's/cat/dog/'      # dog cat
$ echo "cat cat" | sed 's/cat/dog/g'     # dog dog
$ sed 's/IT/Tech/' people.csv             # print edited text (file unchanged)
$ sed 's/chennai/CHN/I' people.csv        # I = ignore case
$ sed 's|/usr/bin|/opt/bin|' paths.txt    # other delimiter avoids escaping /
```

Delete and print lines

```
$ sed '1d' people.csv                 # delete line 1 (header)
$ sed '$d' people.csv                 # delete last line
$ sed '2,4d' people.csv               # delete lines 2 to 4
$ sed '/HR/d' people.csv              # delete lines containing HR
$ sed -n '2,3p' people.csv            # print ONLY lines 2 and 3
$ sed -n '/IT/p' people.csv           # like grep
```

Editing files in place

```
$ sed -i 's/IT/Tech/g' people.csv          # modifies the file!
$ sed -i.bak 's/IT/Tech/g' people.csv      # keep backup people.csv.bak
```

Warning
`sed -i` rewrites the file immediately. Use `-i.bak` until you trust your pattern.

Multiple commands

```
$ sed -e 's/IT/Tech/' -e 's/HR/People/' people.csv
$ sed 's/^/> /' people.csv                 # add a prefix to every line
$ sed 's/[ \t]*$//' file.txt               # strip trailing spaces
```

Practice

- Replace `Chennai` with `Chn` and print the result without changing the file.
- Print `people.csv` without its header and without any `HR` lines.
- Make a backup-safe edit changing `Sales` to `Marketing` in place.

MISSION 5 | +50 XP

Programmable Text: awk

Goal: Process columns, filter by conditions and compute totals.

How awk thinks

awk reads input line by line, splits each line into fields (`$1`, `$2`, ... , `$0` is the whole line) and runs your action for lines that match a condition.

```
awk 'CONDITION { ACTION }' file
```

Variable	Meaning
`$1 $2 ...`	Field values
`$0`	Whole line
`NF`	Number of fields in the line
`NR`	Current line number
`FS` / `-F`	Field separator
`BEGIN{}` / `END{}`	Run before first / after last line

```
$ awk '{print $1}' file.txt                    # first column (space separated)
$ awk -F',' '{print $1, $3}' people.csv          # CSV: name and salary
$ awk -F',' 'NR>1' people.csv                    # skip header
$ awk -F',' '$3 > 50000' people.csv              # salary over 50000
$ awk -F',' '$2=="IT" {print $1}' people.csv     # names in IT
$ awk -F',' 'NR>1 {sum+=$3} END {print sum}' people.csv      # total
$ awk -F',' 'NR>1 {s+=$3} END {print s/(NR-1)}' people.csv   # average
$ awk '{print NR": "$0}' people.csv             # number the lines
$ awk 'END{print NR}' people.csv                 # line count
```

Grouping with arrays

```
$ awk -F',' 'NR>1 {total[$2]+=$3} END {for (d in total) print d, total[d]}' \
    people.csv
IT 171000
HR 82000
Sales 45000
```

Formatted output

```
$ awk -F',' 'NR>1 {printf "%-10s %8d\n", $1, $3}' people.csv
```

Practice

- Print names and cities of employees earning more than 45000.
- Compute the total salary of the IT department.
- Print the number of fields on each line of `/etc/passwd` using `-F:`.

BOSS BATTLE | +150 XP

Boss: Log Detective

Goal: combine everything from this module without hints.

Scenario

A web server log is full of requests. Your manager wants the top visitors, the error count and a cleaned report.

Tasks

1. Create `access.log` with at least 10 lines such as `10.0.0.1 GET /index.html 200`, using several IPs and a few `404` and `500` codes.
2. Count the total number of requests.
3. Count requests with status `404`.
4. List each unique IP with its request count, highest first.
5. Show only lines for status 500 with line numbers.
6. Print the IP and path for every request whose status is not `200` using awk.
7. Create `report.txt` that replaces `GET` with `READ` (keep original untouched) and also prints the result on screen.
8. Compute the number of requests per status code with awk.

Sample solution

```
wc -l access.log
grep -c ' 404$' access.log
awk '{print $1}' access.log | sort | uniq -c | sort -rn
grep -n ' 500$' access.log
awk '$4 != 200 {print $1, $3}' access.log
sed 's/GET/READ/' access.log | tee report.txt
awk '{c[$4]++} END {for (s in c) print s, c[s]}' access.log
```

Check Your Understanding

Q1. What does the | symbol do?

Q2. Difference between > and >>?

Q3. Why sort before uniq?

Q4. Which grep flag inverts the match?

Q5. How do you cut the 2nd comma-separated column?

Q6. What does `sed 's/a/b/g'` do?

Q7. Which sed flag edits a file in place?

Q8. What is `NR` in awk?

Q9. How do you print only line 5 of a file?

Q10. How do you save stdout and stderr together?

Answers

#	Answer
1	Sends one command's output into the next command's input.
2	> overwrites, >> appends.
3	uniq only removes adjacent duplicates.
4	-v
5	cut -d',' -f2
6	Replaces every a with b on each line.
7	-i (use -i.bak for a backup).
8	The current line number.
9	sed -n '5p' file
10	cmd > out.txt 2>&1

Quick Reference Cheat Sheet

Command	Purpose	Example
wc -l	Count lines	wc -l file
grep -i / -v / -c	Search lines	grep -vi error log
cut -d, -f	Pick columns	cut -d, -f1 f.csv
sort -n -r -k	Order lines	sort -t, -k3 -nr f
uniq -c	Count duplicates	sort f | uniq -c
tr	Translate chars	tr a-z A-Z
sed s///g	Replace text	sed 's/a/b/g' f

Command	Purpose	Example
sed -n 'p'	Print lines	sed -n '2,5p' f
awk -F	Fields and maths	awk -F, '{print $1}' f
tee	Save and pass on	cmd | tee out.txt
2>&1	Merge stderr	cmd > f 2>&1

Up next: Module 04
Permissions. Understand rwx, chmod, chown, chgrp and umask to control who can read, write and run files.